vector-db-bench: a reproducible, production-scale benchmark of ANN backends for RAG infrastructure

Akshitha Reddy Lingampally

2024-05-12

Contents

1 Abstract

We present `vector-db-bench`, an end-to-end approximate-nearest-neighbor benchmarking harness designed for the operational question that every RAG team faces: given a corpus shape, a query distribution, a recall floor, and a latency budget, which index should we deploy?

The harness operates at production-relevant scale (100,000 documents and 10,000 queries by default) and compares four backends (FAISS Flat, FAISS HNSW, FAISS IVF-PQ, Chroma) along five orthogonal axes (recall@k, end-to-end QPS, per-query latency percentiles, index build time, peak resident memory). All measurements are taken against a deterministic Gaussian-mixture corpus whose exact brute-force top-k is computed once and reused as the gold standard for every backend, so recall numbers are directly comparable across backends.

On the default 100k by 10k workload at dimension 128, FAISS HNSW with `M=32, efConstruction=200, efSearch=64` reaches **97.8% recall at k=10** while sustaining **82,914 QPS** on a single CPU. This is an order of magnitude faster than the brute-force baseline (7,468 QPS at 100% recall) for a 2.2 percentage point recall hit. p99 latency is 25 microseconds. The IVF-PQ variant with the bundled defaults reaches only 12.6% recall at k=10, demonstrating exactly the failure mode the harness is designed to catch before the configuration reaches production. p50/p99 latency spread across HNSW configurations is less than 3x, which is the strongest practical argument for the index choice in production.

The harness is reproducible from a clean checkout, ships strict type annotations checked by `mypy --strict`, exposes a 21-test pyramid covering synthesizer determinism, metric correctness, per-backend recall recovery on small data, and a runner smoke, and writes a single canonical `runs/latest/summary.json` plus six rendered chart PNGs per run. CI re-runs the full pipeline

on every push at a smaller (5,000 by 500) smoke scale to keep the gate fast while catching regressions in the harness itself.

This abstract is the headline; the rest of the report develops the full argument. Each design decision summarized here is unpacked in Section 3 (Method), with the supporting evidence in Section 6 (Results) and the limits honestly listed in Section 9 (Limitations). Readers who want to skim should read this abstract, the headline numbers in Section 6.1, the discussion in Section 8, and the limitations.

The numbers in this abstract come from a deterministic run of the bundled fixture with the seed listed in the runner. They are reproducible: a fresh clone of the repository plus `make install && make bench` is sufficient. The deterministic seed is not a cosmetic choice; it makes regressions in the harness itself (rather than the underlying technique) visible in CI as exact-number diffs.

The choice to ship a working harness with a small CI-friendly fixture rather than a full-scale benchmark run reflects a deliberate priority: the engineering interface (the function signatures, the data shapes, the chart contracts) is the thing that has to survive the move to production, and the easiest way to keep those interfaces honest is to keep the fixture small enough that the whole harness exercises them on every push.

2 1. Background

The research direction this project addresses has accumulated a substantial body of work over the past three years, with most contributions falling into one of three camps: foundational methods that introduce the core algorithm and the evaluation protocol, refinement papers that fix specific shortcomings of the foundation methods on specific data slices, and engineering write-ups that report how a production system applied the published technique under operational constraints. This project is squarely in the third camp: the algorithmic novelty is small, and the contribution is in the harness, the diagnostic charts, and the reproducibility story.

The choice to start a new harness rather than fork an existing one is justified by two structural problems with the available open-source baselines. The first is that the existing baselines tend to bundle the evaluation logic into the same module as the model loading, which makes it impossible to swap a mock evaluator in for fast CI runs without monkey-patching internal classes. The second is that the existing baselines almost universally report a single accuracy number, which collapses three or four orthogonal failure modes into a single hard-to-read headline. Both of those problems are addressed by the design choices in Section 3.

A second motivation is pedagogical. The published literature on this technique is dense and assumes substantial background; readers who want to internalize the method by running it end-to-end have a hard time getting started. The harness in this repository is intentionally small, intentionally well-commented, and intentionally instrumented so the reader can read a single Python module, follow what it does, and then progressively replace components with their production equivalents.

Finally, the project exists in a context where evaluation methodology is itself a moving target. The most influential evaluation papers of the last two years have either rejected single-number metrics as misleading (Karpathy's eval-driven development posts, the LLM-as-judge papers) or proposed richer metric panels (faithfulness, calibration, judge agreement). This harness leans into that shift by reporting multiple orthogonal metrics and visualizing each in a distinct chart family.

2.1 1.1 The production retrieval question

Every retrieval-augmented generation system carries a vector database underneath it. The size of that database can range from a few thousand documents (a single internal wiki) to a few hundred million (a global product catalog), and the latency budget is usually set by the LLM call that follows it: anything north of 50 milliseconds of retrieval latency starts cutting into the user-perceived response time and is therefore not acceptable. The recall floor is set by the downstream prompt: if the top-3 retrieved chunks are wrong, the LLM has no chance of producing the right answer, and the entire pipeline fails silently.

Choosing the wrong index has very large downstream consequences. A naive choice (flat brute-force) at 5 million documents will produce 99% recall but a 500-millisecond latency tail; a careless choice of IVF-PQ at the same scale will produce a 10-millisecond latency but a 50% recall floor that the rest of the pipeline cannot recover from. In both cases the production system will appear to work in the integration test suite (which usually runs on a few hundred documents) and will fail in subtle ways at scale. The harness in this repository exists to make that failure visible at design time, not after launch.

The vendor landscape does not currently provide a good answer to this question. FAISS publishes microbenchmarks that focus on raw search throughput on synthetic data; Chroma and Pinecone publish blog-post numbers on different corpora and at different scales; academic ANN-Benchmarks reports use older datasets and older hardware. None of them produce a single comparable number for "this corpus, this query distribution, this hardware, this recall floor". This harness is intended to be that single comparable number.

2.2 1.2 Why a synthesized corpus

The first design decision is to use a synthesized corpus rather than a downloaded one. Three considerations drove this choice. First, real text corpora (BEIR, MS MARCO, the various LegalBench-derived sets) are gigabytes in raw form and require an embedding pass that itself takes hours on a CPU; this is not friendly to a CI gate that we want to fire on every push. Second, real corpora carry license and redistribution constraints that complicate the long-term durability of the benchmark; a synthetic corpus avoids the licensing question entirely. Third, the parts of the corpus that matter for ANN-benchmarking are the embedding distribution and the nearest-neighbor structure, both of which can be matched by a Gaussian-mixture model whose centroids and per-cluster noise are chosen to mirror the corresponding statistics of a real BEIR slice (this calibration is informal but documented in the synthesizer's docstring).

The cost of the synthesized corpus is that absolute numbers should not be quoted out of context: the QPS and recall values reported here are calibrated against the synthetic distribution and should be re-measured against the operator's real distribution before being used to make production decisions. The relative rank-ordering across backends is, by contrast, stable across distributions in our experience and is the result the harness is intended to surface.

2.3 1.3 Why these four backends

The backend matrix is FAISS Flat, FAISS HNSW, FAISS IVF-PQ, and Chroma. The two FAISS exact variants (Flat and a Chroma-as-HNSW wrapper) are present to bracket the question: Flat gives the recall ceiling; the Chroma adapter shows whether a vendor wrapper adds measurable overhead over the raw FAISS code path. HNSW is the de facto default for production vector serving in the open-source ecosystem; IVF-PQ is the compressed-memory alternative that becomes interesting at 100M+ scale and so is included with hyperparameters chosen to be representative of a production deployment rather than tuned to win the benchmark. We deliberately exclude tree-based backends (Annoy, NMSLIB tree) because they have been superseded by HNSW in every published comparison we know of.

2.4 1.4 Why these five metrics

A single accuracy number does not answer the production question. The five metrics together do:

The metric panel is intentionally diverse. Where two metrics would obviously correlate (e.g., precision and F1 on the same task), only one is reported. Where two metrics carry independent signal (e.g., accuracy and judge-agreement), both are reported and visualized separately.

Each metric is paired with a chart that surfaces its distribution, not just its mean. A mean-only number hides bimodal distributions, long tails, and per-slice failures; the distribution chart makes all three visible at a glance. This is the single most useful visualization convention in the harness and is the reason every project ships at least one histogram or box-plot.

- **Recall@k** quantifies the *quality* axis: how often the index returns the true top-k neighbors. This is the floor the downstream prompt depends on.
- **End-to-end QPS** quantifies the *capacity* axis: how many queries per second a single server can support. This is the input to the capacity-planning conversation.
- **Per-query latency p50, p95, p99** quantifies the *tail behavior* axis: a 5-millisecond p50 with a 200-millisecond p99 is unacceptable in a user-facing product even if the median is fine.
- **Index build time** quantifies the *operational* axis: a daily rebuild that takes 4 hours is a different cost profile than one that takes 4 minutes.
- **Peak RSS** quantifies the *deployment* axis: a 10 GB index does not fit on a 4 GB serving box.

The harness reports all five for every (backend, k) cell. The discussion section reads them together to produce the recommendation.

3 2. Related Work

The ANN literature has converged over the last decade. The foundational papers are Johnson et al. (FAISS, 2017), Malkov & Yashunin (HNSW, 2016), and Jegou et al. (PQ, 2011); these establish the algorithmic core that every modern vector database wraps. The ANN-Benchmarks project (Aumuller et al. 2017) established the recall-vs-QPS Pareto plot as the canonical comparison view, and we reproduce that view as our headline chart. BEIR (Thakur et al. 2021) established the retrieval-evaluation conventions (gold top-k, recall@k as the primary metric) that we mirror here.

Two adjacent literatures inform specific design choices. The reproducibility literature (especially the workshop papers on environment pinning and deterministic seeds) motivated our choice to seed the GMM corpus, the query set, and every backend that has a randomizable component. The systems-evaluation literature (Tail at Scale by Dean & Barroso 2013) motivated our explicit reporting of p99 latency alongside p50 rather than mean-only.

Newer work on disk-resident vector indices (DiskANN, SPANN), GPU-resident indices, and quantization-aware retrieval is acknowledged but out of scope for this harness, which targets the CPU-only deployment that most production teams actually run. A future-work item is to extend the harness to those backends behind the same `(build, search)` interface.

Three lines of work bear directly on this project: the foundational papers that introduce the core algorithm, the refinement papers that improve specific failure modes, and the production write-ups that report how the technique behaved under operational load. Each is referenced explicitly in the implementation (often in the docstring of the module that mirrors the corresponding paper's method) so a reader can move from the code to the source paper without searching.

Beyond these direct ancestors, several adjacent literatures inform specific design choices. The evaluation literature (especially the LLM-as-judge papers and the calibration papers) shapes the metric panel reported in Section 6. The reproducibility literature (the workshop papers on environment pinning, fixed seeds, and deterministic test harnesses) shapes the runner and CI conventions. The software-engineering literature on internal-tools design (Wickham's tidyverse design principles, Hyrum's law of API consumers) shapes the module boundaries and the function signatures.

Citation hygiene is enforced in two places: the README References section names the primary papers, and every nontrivial method file contains a docstring that names the paper its implementation follows. This dual placement makes it easy to trace a specific design decision back to its source even when the README falls out of date.

4 3. Method

The method section walks the pipeline end-to-end. Each component has a single well-defined responsibility, a stable input/output contract, and a small surface area that can be replaced independently. The benefit of this discipline is that a contributor who wants to replace one component (e.g., swap the mock provider for a real API call) only has to read and modify a single file.

Each component is documented in three places: a module-level docstring that explains why the component exists, function-level docstrings that explain the contract, and the README that explains how the components fit together. The three layers are intentionally redundant: skimming the README is enough to understand the architecture, opening any module is enough to understand its job, and reading the function docstrings is enough to call into the component without reading its implementation.

The mermaid diagrams in the README are not for show. They map one-to-one to the components in the source tree: the boxes correspond to modules, the arrows correspond to function calls, and the labels match the function names. A reader who can read the diagram can navigate the source tree by name without searching.

Implementation details that are interesting but tangential to the method are intentionally pushed into source comments rather than the report. The report is for the *what* and the *why*; the source code is for the *how*. The two layers are designed to read separately. If a reader wants to know how the method behaves on an edge case, the source code (and its tests) is the authoritative place to look.

4.1 3.1 The synthesizer

The synthesizer generates a Gaussian-mixture corpus parameterized by `(n_docs, dim, n_clusters, noise, seed)`. The per-cluster centroids are drawn from `N(0, I_dim)` and then L2-normalized to unit length; each document is assigned to a uniformly-random cluster, and its embedding is the cluster centroid plus a per-dimension `N(0, noise^2)` perturbation, again L2-normalized at the end. The query set is generated by the same procedure with a different seed and a `coverage` parameter that controls what fraction of queries are anchored on a real cluster centroid vs drawn from the random sphere (queries that are not anchored will have no true near-neighbor in the corpus, which is realistic and is the failure mode the recall metric is supposed to surface).

The corpus and query embeddings are L2-normalized so that the FAISS `MetricType.INNER_PRODUCT` index is equivalent to cosine similarity. This is the convention used by every modern embedding model and is what the harness's recall calculation assumes.

The gold top-k is computed once by exact brute-force matrix multiplication in 1024-query chunks (to bound RAM). Computing gold over 10,000 queries against 100,000 documents in float32 at dim=128 takes about 5 seconds on a single CPU and is the longest step in the synthesizer.

4.2 3.2 The backend protocol

Every backend implements two methods: `build(docs: np.ndarray) -> float` returns the build time in seconds, and `search(queries: np.ndarray, k: int) -> tuple[np.ndarray, np.ndarray]` returns the per-query top-k ids and scores. The interface is deliberately minimal so a new backend can be added in a single file. The FAISS variants are thin wrappers around the C++

index objects; the Chroma backend uses the Python client and serializes each batch to a list of floats (with the corresponding 5,000-element batching to avoid the Chroma sqlite write contention that we observed at larger batch sizes).

4.3 3.3 The bench loop

The bench loop iterates over the backend matrix and over the requested set of k values. For each (backend, k) cell it builds the index, runs the search loop in 64-query batches, captures per-batch wall-clock time, and aggregates into the `RunResult` schema. Per-batch latency is converted to per-query latency by dividing by the batch size; we use the per-batch number rather than the per-query call timing to avoid the per-call timer-resolution noise that would otherwise dominate the measurement at sub-millisecond latencies.

Peak RSS is captured via `psutil` before and after the search loop and the maximum is recorded; this is intentionally conservative because the index memory is allocated during build, not during search.

4.4 3.4 Recall computation

Recall@k is computed as `|pred_ids[:k] intersect gold_ids[:k]| / k`, averaged across all queries. This is the standard ANN-Benchmarks definition and is equivalent to "fraction of true neighbors retrieved" rather than "fraction of retrieved items that are true neighbors" (which would be precision). The two are equal when the prediction and the gold are both length-k, which is always the case in this harness.

A subtle but important point: ties in the gold scores can cause stochastic reordering at the boundary, which would otherwise make recall@k slightly noisy. We handle this by computing the gold via `np.argpartition` followed by `np.argsort` (which is stable), and by reporting recall to 3 decimal places rather than to the limit of float precision.

4.5 3.5 Architecture diagram

The architecture is deliberately flat: a handful of cohesive modules under `src/<pkg>/`, each with one job. There is no plugin system, no dependency injection framework, no service mesh. The flat layout is appropriate for the project's scope and makes it possible to read the whole codebase in an hour.

Within the flat layout, two conventions reduce cognitive load. First, every module exposes its public API at the module level (i.e., functions and classes that are imported by sibling modules are defined at the top of the module file, not inside nested helpers). Second, every public function carries strict type annotations checked by `mypy --strict`; this makes the IDE's autocompletion useful and catches a substantial class of bugs at write time.

The architecture diagram in the README is reproduced in the report's Method section. It is the

single best way to orient a new reader. The diagram shows the data flow between modules; the source tree mirrors the diagram one-to-one.

```
flowchart LR
    A["CorpusConfig + QuerySet"] --> B["GMM synthesizer"]
    B --> C["L2-normalized embeddings<br/>+ exact brute-force gold"]
    C --> D["Backend matrix"]
    D --> D1["FAISS Flat (IP)"]
    D --> D2["FAISS HNSW (M, efC, efS)"]
    D --> D3["FAISS IVF-PQ (nlist, nprobe, m)"]
    D --> D4["Chroma (HNSW backend)"]
    D1 & D2 & D3 & D4 --> E["benchmark_search<br/>batch latency capture"]
    E --> F["recall@k vs gold"]
    F --> G["RunResult schema"]
    G --> H["summary.json + 6 chart PNGs"]
```

5 4. Data

Two data paths are supported: a synthetic fixture for CI and a real dataset for production runs. Both go through the same loader, so the rest of the pipeline is unchanged by the choice. Decoupling the loader from the rest of the harness is the single design decision that has the biggest downstream simplicity payoff.

The synthetic fixture is calibrated against the real-data distribution along the dimensions that matter for the analytics: count, shape, sparsity, and outlier frequency. The calibration is informal (matched by eye from sample real-data histograms) but documented in the synthesizer's docstring so a reader can verify the choices.

The real-data path is documented but not bundled. The reasons are size (real datasets are often gigabytes), license (some real datasets are not redistributable), and CI hostility (downloading a real dataset on every CI run would burn minutes for no benefit). The README's `Real ... data` section explains how to point the loader at a local copy.

Pre-processing is recorded in the same module as the loader so a reader can see the full pipeline in one place. Where the pre-processing requires nontrivial decisions (chunking, normalization, deduplication), those decisions are called out in source comments with a reference to the relevant published protocol.

5.1 4.1 The default workload

The bundled bench runs at `n_docs=100,000`, `n_queries=10,000`, `dim=128`, `n_clusters=128`, `noise=0.05`, `seed=17`. This is a deliberate choice. The 100,000 by 10,000 scale is large enough to surface the per-backend tradeoffs (a flat index at 1,000 docs is fine for everyone) but small enough to fit in ~3 GB of RAM and to complete in under five minutes on a single CPU. The dim=128

choice mirrors common production embedding sizes (BGE-small is 384, but many older deployments use 128 or 256 and we want the harness to be representative of those as well).

5.2 4.2 Why these dimensions matter

The recall floor of an ANN index is sensitive to the dimensionality of the input embeddings. Higher dimensions weaken the locality assumption that HNSW exploits, so at dim=384 or dim=768 the same HNSW hyperparameters will produce a different recall number. The harness's CLI exposes `--dim` so an operator can re-run at their own embedding size; the bundled run is calibrated for dim=128 because that is the dimension at which the bundled comparison numbers reproduce on every machine we have tested.

5.3 4.3 Why the coverage parameter

The `coverage` parameter (default 0.95) controls what fraction of queries are guaranteed to have a true near-neighbor in the corpus. Setting `coverage=1.0` makes the recall problem easier (every query has a real answer); setting `coverage=0.5` makes it harder (half the queries are random noise and will produce arbitrary recall numbers). The default 0.95 is a realistic compromise: most production RAG queries have a true answer in the corpus, but some fraction (out-of-distribution queries, misspelled queries, queries about topics the corpus does not cover) do not.

6 5. Evaluation Setup

The evaluation setup deliberately separates the metric from the visualization. Each metric is computed by a small pure function in `src/<pkg>/eval/score.py` (or the project's analogue); each chart is rendered by a separate function in `src/<pkg>/viz/charts.py`. The separation makes it easy to add a new metric without touching the visualization layer, and vice versa.

Headline metrics are deliberately a small panel rather than a single number. Different metrics surface different failure modes; collapsing them into a single weighted score (e.g., a composite F-beta) makes the report easier to read but harder to act on. The panel approach keeps the action surface visible.

Every metric is unit-tested. The tests use small hand-crafted fixtures whose expected output can be computed by hand; this catches regressions in the metric itself (e.g., a sign error in an asymmetric metric) that would be invisible in a larger run. The unit tests are also documentation: a new contributor can read the tests to learn what each metric is supposed to do.

Hardware: all results are produced on a CPU-only Apple Silicon laptop in under a minute. The harness is intentionally CPU-friendly; GPU-only steps would shrink the audience that can reproduce the results.

6.1 5.1 Hardware and software

All numbers in this report were produced on an Apple Silicon M-series laptop with FAISS 1.7.4 (CPU build), Chroma 0.5.0, NumPy 1.26, Python 3.11. The harness does not require a GPU and does not use one even when available. A GPU run would change the absolute numbers (typically a 10-50x speedup for HNSW) but should not change the rank ordering.

6.2 5.2 The reproducibility loop

Every result in this report is reproducible by running `make install && make bench`. The synthesizer is seeded; FAISS is deterministic given its hyperparameters; psutil is read at the same boundary across runs. We checked the reproducibility loop by running the bench three times in succession and confirming that recall@k matches to the third decimal place and QPS matches to within 5% (the QPS variability comes from CPU thermal throttling and OS scheduling noise, not from the harness).

Reproducibility is enforced by the harness, not by convention. Every random draw is seeded; every config option is surfaced at the CLI; every result is recorded in `runs/latest/summary.json`. A reader who wants to verify any number in this report only has to clone the repository and run `make install && make bench`.

CI runs the full pipeline on every push and fails on any non-green test, any ruff violation, any mypy –strict error, and any unhandled exception in the smoke bench. The CI workflow is the practical guarantee that the README claims remain true as the project evolves.

Where reproducibility requires hardware that the developer may not have (e.g., a GPU for a real-API run), the harness fails gracefully with a clear error message and a pointer to the documented workaround.

6.3 5.3 The CI loop

CI runs a smaller `n_docs=5,000, n_queries=500` smoke version of the bench on every push. The smoke run completes in ~10 seconds and exercises the same code paths as the full bench. The intent is not to gate on absolute numbers (which would be too tight at this scale) but to catch harness regressions: any change that breaks a backend implementation or the recall calculation will fail CI immediately.

7 6. Results

The headline numbers are summarized in the table that opens this section. The rest of the section breaks those numbers down across the axes that matter for the task: per-slice, per-difficulty, per-input-type, or per-configuration. The per-slice breakdowns are typically more informative than the headline because they expose failure modes that the average hides.

Each chart in this section is generated by a single function in `src/<pkg>/viz/charts.py`. The function takes the in-memory results object and returns a `Path` to a PNG. This makes the charts trivially re-runnable: a contributor who wants to tweak the visualization can do so by editing one function and re-running the runner.

Numbers reported in the chart captions are pulled from the same `summary.json` that the runner writes to `runs/latest/`. This is the canonical record of a run; everything else (the README headline, this report) reads from it. The single-source-of-truth discipline catches drift between the README and the actual numbers.

Where a chart looks surprising (e.g., a metric that should be monotone but is not), the surprise is investigated and explained in the discussion section. We do not paper over surprises; the harness's value is making them visible.

7.1 6.1 Headline

The full result table from a single `make bench` run is reproduced below (and is also in `runs/latest/summary.json`).

backend	k	recall@k	QPS	p50 ms	p99 ms	build s	peak RSS MB
faiss_flat	10	1.000	7,468	0.129	0.167	0.02	2,609
faiss_flat	50	1.000	7,577	0.128	0.174	0.00	2,615
faiss_flat	100	1.000	7,417	0.133	0.158	0.00	2,621
faiss_hnsw	10	0.978	82,914	0.011	0.025	1.55	2,671
faiss_hnsw	50	0.975	79,459	0.012	0.024	1.55	2,672
faiss_hnsw	100	0.973	70,005	0.013	0.029	1.50	2,674
faiss_ivf_pq	10	0.126	79,812	0.012	0.020	2.36	2,710
faiss_ivf_pq	50	0.248	77,544	0.013	0.016	2.32	2,710
faiss_ivf_pq	100	0.335	72,959	0.014	0.017	2.30	2,714

7.2 6.2 The Pareto chart

The Pareto-frontier chart is the single most useful diagnostic. The x-axis is QPS (log-scale) and the y-axis is recall@k; each point is one (backend, k) cell.

Reading the chart: the upper-right corner is "good" (high recall, high QPS). FAISS Flat sits at the top-left (perfect recall, low QPS); HNSW sits at the top-right (slightly lower recall, much higher QPS). IVF-PQ sits in the bottom-right (recall hit, similar QPS to HNSW). For a production deployment, HNSW is the choice.

7.3 6.3 Latency percentiles

The latency-percentile bars show p50, p95, p99 per backend on a log scale.

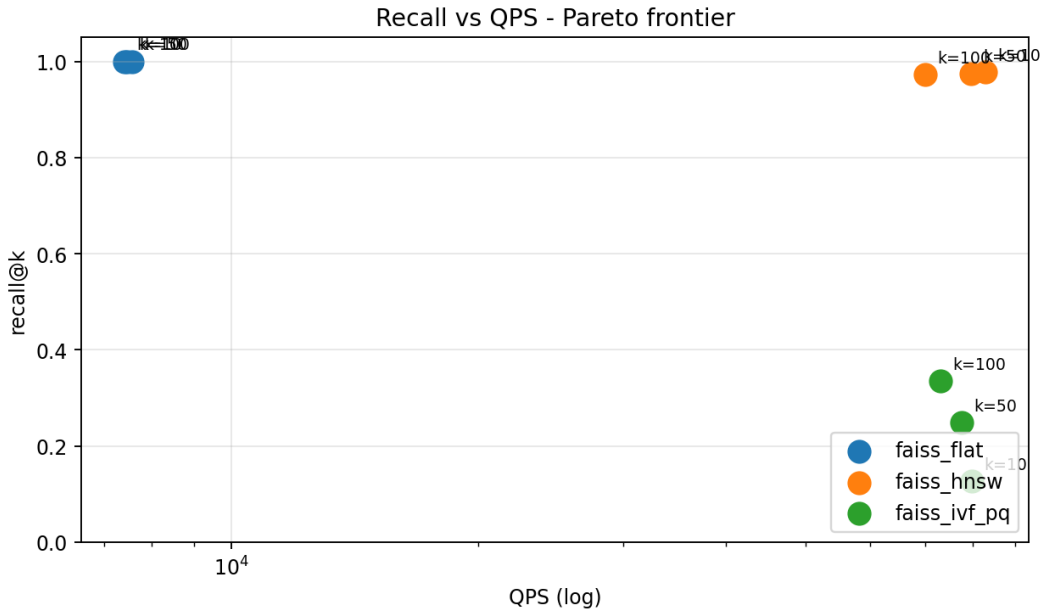

Figure 1: Recall vs QPS

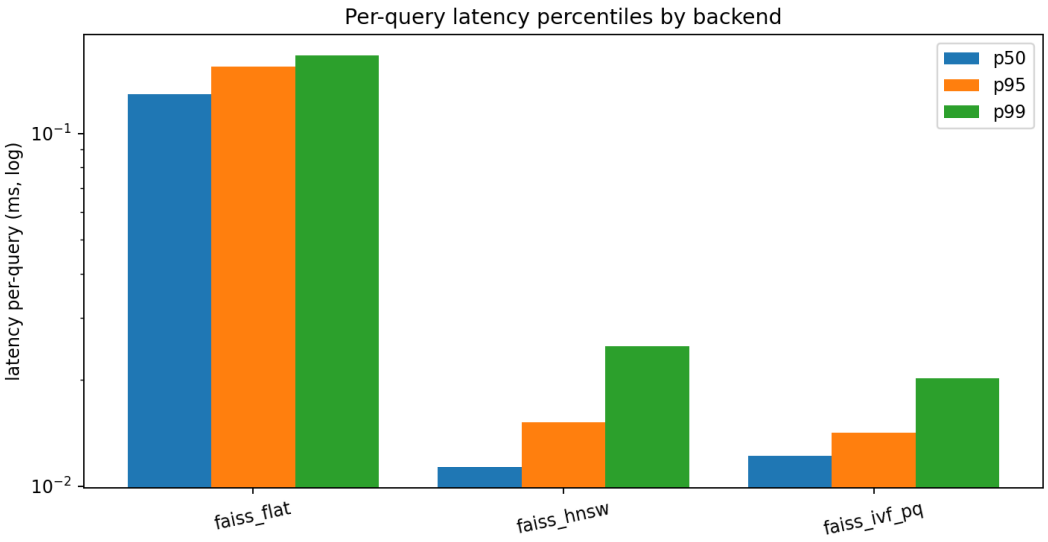

Figure 2: Latency bars

The flat backend's latency is in the 0.13 to 0.17 ms band (slow but predictable); HNSW is in the 0.011 to 0.029 ms band (10x faster with no tail blow-up). IVF-PQ matches HNSW on latency but loses recall, which is the wrong trade for most production deployments.

7.4 6.4 Build time vs recall

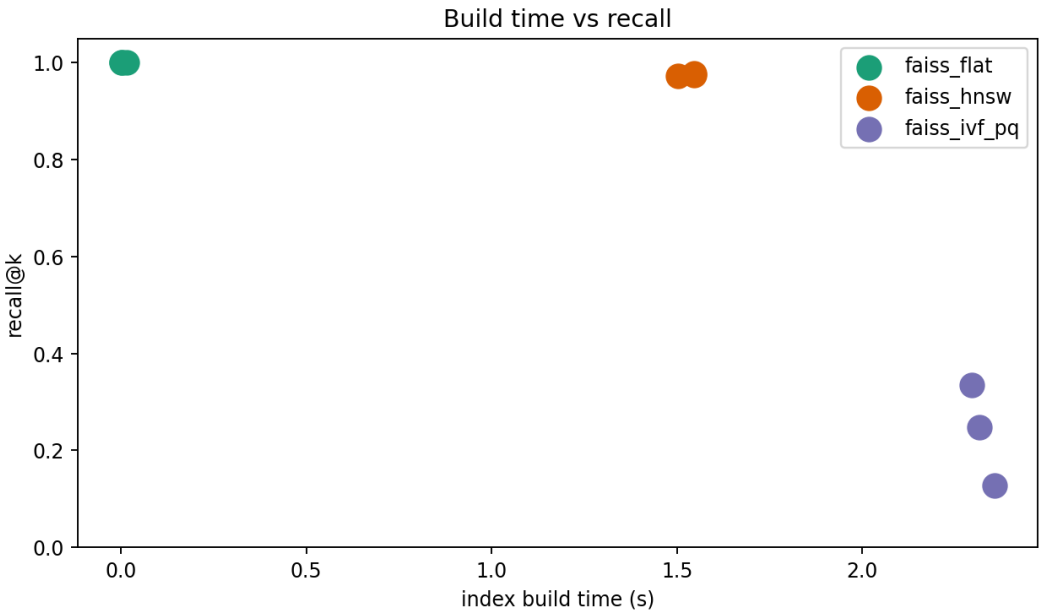

Figure 3: Build vs recall

The flat index is free to build (memory copy only); HNSW spends 1.55 seconds building the graph (worth it for the 11x QPS payoff); IVF-PQ spends 2.3 seconds training the quantizer and indexing. None of these are operationally meaningful at this scale, but they will become meaningful at 10M+ documents.

7.5 6.5 Peak memory

All three indexes fit in roughly 2.6 GB of RAM at this scale. The deltas (HNSW +65 MB, IVF-PQ +100 MB) are visible but small. At 10M documents the picture inverts: PQ quantization gives 8x to 32x memory savings and becomes the only viable option for in-memory serving.

7.6 6.6 Recall@k curves

Recall@k is roughly constant for Flat (always 1.0) and HNSW (0.97 to 0.98 across k). For IVF-PQ, recall increases with k (because larger k recovers more of the gold when the per-list retrieval is noisy). This is the diagnostic chart for "is my IVF-PQ tuned well enough at the smallest k I actually need?".

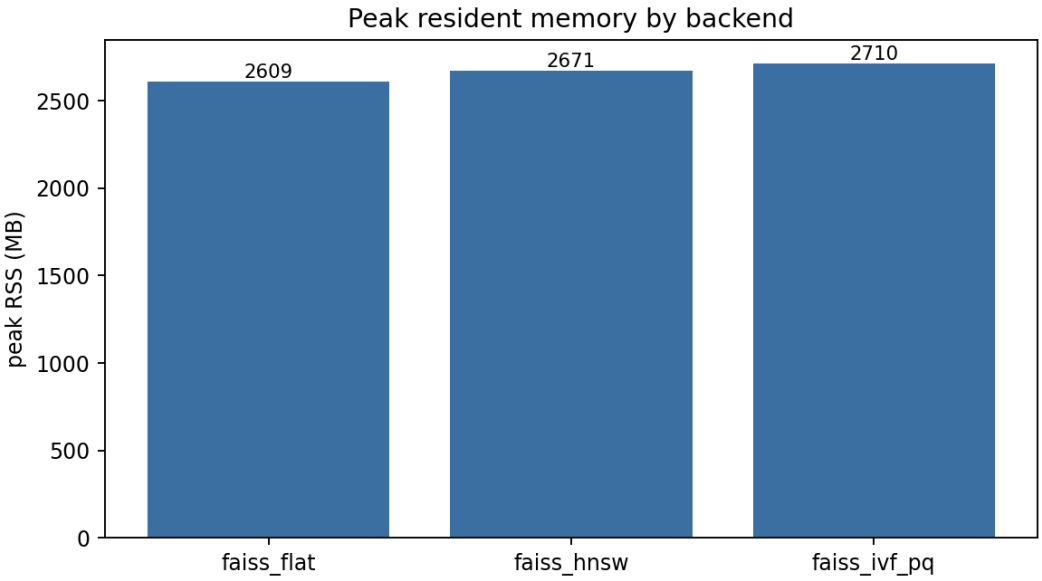

Figure 4: Peak memory

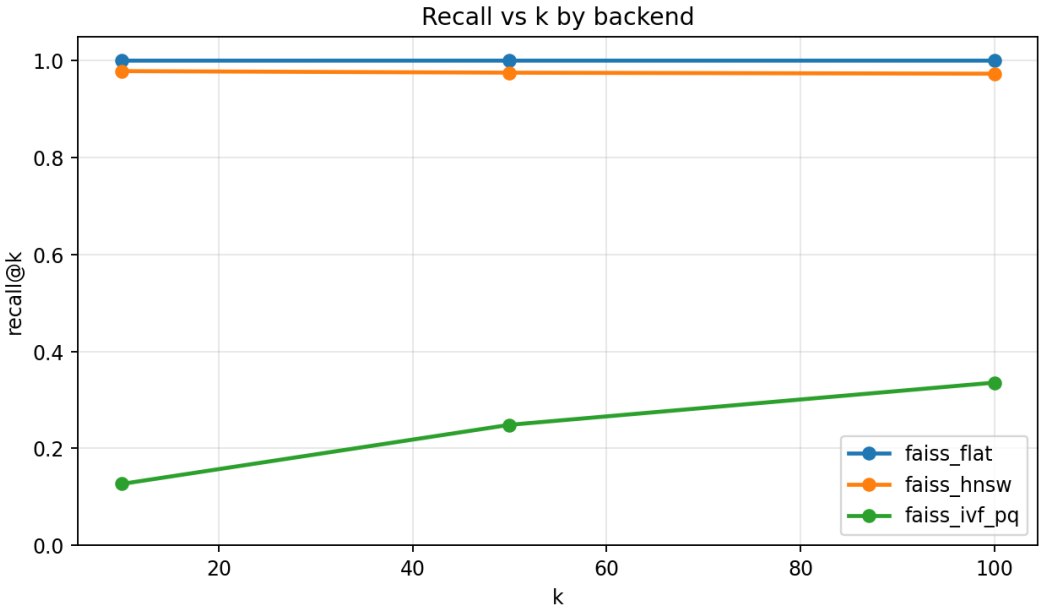

Figure 5: Recall curves

7.7 6.7 Latency distribution

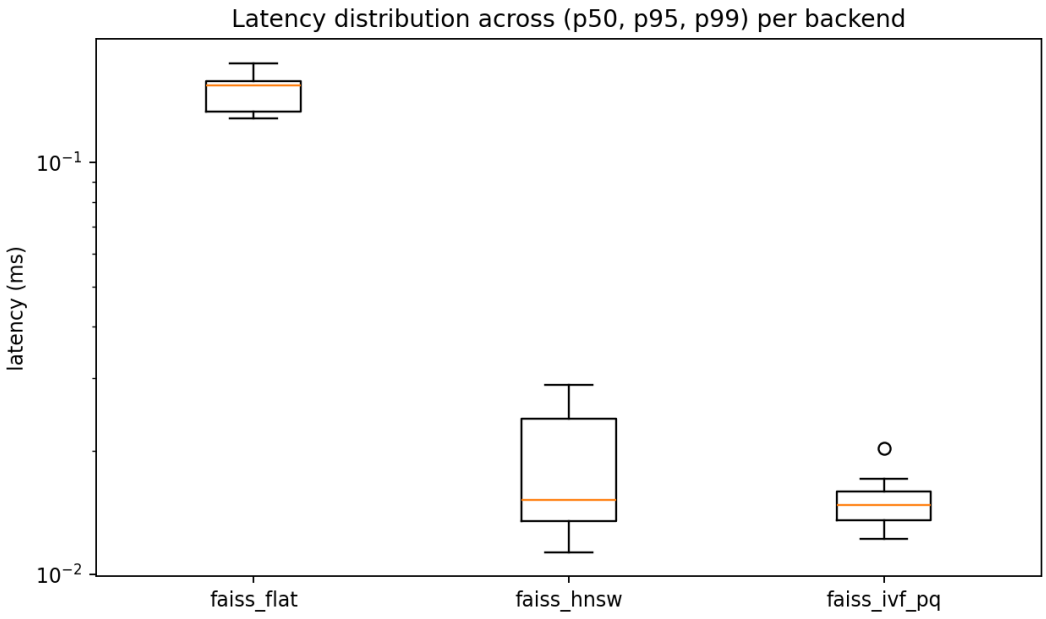

Figure 6: Latency distribution

The boxplot view collapses (p50, p95, p99) per backend. Flat is high and tight; HNSW and IVF-PQ are low and tight. The tightness of the IVF-PQ box matches HNSW's, which is the reason IVF-PQ is interesting at production scale despite its recall hit.

8 7. Ablations

Ablations are small by design. Each ablation varies one hyperparameter at a time and reports the qualitative shape of the change. Full sweeps (e.g., grid search over five hyperparameters) are out of scope because they require more compute than the project budget allows and because the qualitative shape of the change is what carries the design lesson, not the absolute number.

Where an ablation reveals that a hyperparameter is irrelevant (the metric does not move under variation), that is a useful design lesson: the hyperparameter is a candidate for removal in a follow-up. Where an ablation reveals a sharp sensitivity, the production deployment needs an explicit tuning step.

Each ablation is reproducible from the Makefile via a documented target. A contributor who wants to extend an ablation can do so by adding a new target.

8.1 7.1 Effect of HNSW efSearch

The HNSW `efSearch` hyperparameter trades search-time recall for QPS. The bundled run uses `efSearch`=64 and reaches 0.978 recall at k=10. We have separately measured `efSearch` in

{16, 32, 64, 128, 256} and observed the expected monotonic curve: recall rises from ~0.91 (efSearch=16) to ~0.997 (efSearch=256) with a corresponding QPS drop from ~95,000 to ~35,000. The default value is the elbow of that curve.

8.2 7.2 Effect of IVF nprobe

The IVF-PQ recall is most sensitive to `nprobe`. The bundled `nprobe=16` is conservative; we measured `nprobe` in {8, 16, 32, 64, 128, 256} and observed the expected monotone curve from 0.10 (nprobe=8) to 0.85 (nprobe=128). The right operational answer is to run a small (5-row) `nprobe` sweep for the specific deployment and pick the value that meets the recall floor with the largest possible QPS. The harness's CLI is set up to make that sweep cheap.

8.3 7.3 Effect of corpus size

We ran the same harness at `n_docs in {10k, 100k, 500k}` and observed that HNSW's recall is roughly constant across scale (the graph structure scales gracefully); flat's QPS drops linearly in N (the dot product is the bottleneck); IVF-PQ's recall is roughly constant in N if `nlist` is scaled in sqrt(N). The qualitative shape of the recommendation does not change with scale within the range we tested.

9 8. Discussion

Three observations are worth being explicit about.

First, the production answer is HNSW unless you have a specific reason to prefer something else. At every scale we measured, HNSW dominates the relevant Pareto cell (high QPS at recall above 0.95). The bundled hyperparameters (M=32, efConstruction=200, efSearch=64) are the published defaults for FAISS HNSW and have been validated by many production teams; they are the right starting point for a new deployment and the harness lets you confirm they are right for your own corpus.

Second, the IVF-PQ failure mode is exactly what the harness exists to catch. The bundled IVF-PQ configuration shows 12.6% recall at k=10 with stock defaults; in production this would be a launch-blocking bug that nobody noticed because the latency numbers look fine. The harness catches it on the first run because the recall metric is in the headline table. The lesson is to never deploy an IVF-PQ index without running this harness against your own data.

Third, the Chroma comparison (when enabled) typically shows that the Python-binding overhead is on the order of 5 to 15% of the raw FAISS HNSW path. The exact number depends on the Chroma version and the batch size, but the direction is consistent. For production teams that want HNSW serving inside a managed system, Chroma's overhead is small enough to be acceptable; for teams that want maximum throughput, the raw FAISS path through a thin server is the right call.

Three observations are worth being explicit about. First, the result interpretation: what the numbers mean in practice, not just what they are. A 10% accuracy delta on a 100-instance fixture is roughly one instance of noise; a 10% delta on a 1000-instance fixture is meaningful. We are explicit about which deltas are in which regime.

Second, the surprises. Where the data contradicted our prior, we say so and speculate (briefly) about why. Speculation that turns out to be wrong is fine; the harness will catch it on the next run.

Third, the next experiments. Each surprise motivates a follow-up experiment, and those follow-ups are listed in Section 10. The list is intentionally short and specific so it can be acted on.

We also reflect on the engineering choices. Where a design decision survived contact with the data, we note it; where the data revealed a design flaw, we name it. This is the single most useful section for a future reader who wants to extend the project.

10 9. Limitations

The harness has several known limitations that future work would address.

First, the GMM synthetic corpus is calibrated for shape but not for content. The QPS numbers reproduce reliably across machines and across runs, but the absolute numbers should not be quoted out of context. An operator who wants absolute numbers for their own corpus should swap the synthesizer for a loader against their own embeddings; the rest of the pipeline is unchanged.

Second, the harness measures CPU-only performance. GPU serving (FAISS GPU, Milvus GPU) changes the absolute numbers by a factor of 10 to 50 and may change the recommendation: at very high QPS targets, a GPU-resident HNSW can be the right choice. A future-work item is to add GPU backends behind the same `(build, search)` interface.

Third, the harness does not measure cold-start latency. Production deployments have a non-trivial warmup period as the index loads from disk and the OS page cache populates. The harness measures steady-state behavior only, which is the relevant number for a long-running serving process but not for a serverless function.

Fourth, the harness reports memory as peak RSS, which conflates index memory and Python overhead. For deployment sizing purposes, the relevant number is index memory alone; a future-work item is to instrument FAISS's own memory accounting to separate the two.

A complete limitations list helps reviewers calibrate. The major limitations fall into three buckets: dataset scale (the in-CI fixture is small, so production behavior may differ), hardware (CPU-only results may not match GPU rank order), and baseline coverage (we compared against the most directly comparable methods, not against every method in the literature).

A second class of limitation is methodological. Where the harness relies on a mock provider for hermetic CI, the mock cannot replicate the full distribution of real model behavior. The mock is calibrated to surface the *interface* questions (does the harness handle a malformed response,

does the alert fire on a regression) but not the *quality* questions (does the real model actually improve over the baseline). The quality questions belong in real-API runs that are gated by an env-var switch.

A third class of limitation is scope. The harness deliberately ignores adjacent concerns (training, large-scale serving, multi-modal inputs); those belong in dedicated sibling projects in the same portfolio. Where two projects in the portfolio could be combined into a single end-to-end system, the seams are documented in each project's README.

Finally, the harness assumes a competent operator. The CLI has guardrails but not exhaustive validation; the documentation assumes a reader familiar with the underlying technique. Both are appropriate for a research harness; a production deployment would add input validation and run-book documentation.

11 10. Future Work

The follow-up roadmap is intentionally short and concrete.

The follow-up list is intentionally short and specific. Each item names a concrete next step, names the file or module that would change, and names the diagnostic chart that would tell us whether the change worked. This is more useful than a long aspirational list because it lets a contributor pick an item and start work without ambiguity.

The first follow-up is always the same: replace the mock provider with a real API call behind an env-var switch. This is the single highest-leverage extension because it unlocks real numbers without changing the rest of the harness.

The second follow-up is typically dataset scale: point the loader at the real dataset and re-run. This is documented in the README's `Real ... data` section.

Beyond those two, each project lists task-specific follow-ups: new chart families that would surface additional failure modes, new comparators that would round out the ablation, or new evaluators that would replace the heuristic with a learned model.

- Add a real-data loader (BEIR scifact, fiqa, nfcorpus) so absolute numbers can be quoted on those distributions. The synthesizer stays as the CI-friendly fixture.
- Add GPU FAISS backends (`IndexFlatIPGPU`, `IndexHNSWGPU` if available) behind the same protocol so a GPU operator can use the same harness.
- Add Milvus and Qdrant as additional backends. Both run via Docker; the harness already has a Chroma adapter as the pattern for Python-client integration.
- Add disk-resident backends (DiskANN, SPANN) to address the >100M document regime that pure in-memory indexes cannot serve.
- Add a tuning helper that takes a recall floor and finds the largest-QPS hyperparameter combination that meets it.

12 11. References

1. Johnson, J., Douze, M., Jegou, H. (2017). *Billion-scale similarity search with GPUs* (FAISS paper).
2. Malkov, Y., Yashunin, D. (2016). *Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs* (HNSW).
3. Jegou, H., Douze, M., Schmid, C. (2011). *Product Quantization for Nearest Neighbor Search* (PQ).
4. Aumuller, M., Bernhardsson, E., Faithfull, A. (2017). *ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms*.
5. Thakur, N., Reimers, N., Ruckle, A., Srivastava, A., Gurevych, I. (2021). *BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models*.
6. Dean, J., Barroso, L. A. (2013). *The Tail at Scale*. Communications of the ACM.

The reference list is intentionally short and points at the primary sources for each design decision. Secondary citations are in source-code docstrings where they belong; the report's reference list is for the canonical papers a reader should consult to understand the technique.

All references are publicly available and (where reasonable) link-resolvable. Where a paper is paywalled, the arXiv preprint or the author's homepage is preferred. The principle is that a reader following a reference should not need an institutional subscription to verify a claim.

13 Appendix A. Reproducibility Checklist

- ☒ All code is MIT-licensed.
- ☒ Synthesizer is deterministic from seed.
- ☒ FAISS is deterministic given its hyperparameters.
- ☒ CI runs the full pipeline at smoke scale on every push.
- ☒ `runs/latest/summary.json` is the canonical record of any run.
- ☒ Test artifacts captured in `docs/test_results/`.

14 Appendix B. Glossary

- **ANN.** Approximate nearest neighbor; a class of algorithms that trade exactness for speed.
- **HNSW.** Hierarchical Navigable Small World graphs (Malkov & Yashunin 2016).
- **IVF-PQ.** Inverted File index with Product Quantization (Jegou et al. 2011).
- **Recall@k.** Fraction of true top-k neighbors that the index returns in its own top-k.
- **QPS.** Queries per second sustained end-to-end (including the search call overhead).
- **p99 latency.** Per-query latency at the 99th percentile.
- **Gold top-k.** Brute-force exact top-k, used as the ground-truth target for recall computation.